PONTIFICIA UNIVERSIDAD CATÓLICA DEL ECUADOR

Desarrollo de Plataformas Web

INFORME TÉCNICO Y REGISTRO DE EVIDENCIAS

Unidad 6: Seguridad, Autenticación (JWT), Control de Acceso (RBAC), CORS, HTTPS, Sanitización e Integración Cliente-Servidor

Asignatura:	Desarrollo de Plataformas
Unidad Académica:	Unidad 6 - Seguridad Integral
Entorno Evaluado:	Node.js v24.19.0 / Express / Zod / Prisma ORM

Informe Técnico – Tarea 6

Seguridad, Autenticación, Autorización e Integración Cliente-Servidor en la Plataforma de Incidentes

1. Introducción

La plataforma de incidentes de las unidades anteriores ofrecía una API REST en Node.js y Express, con persistencia en MySQL mediante Prisma, validación con Zod y manejo centralizado de errores. Sin embargo, la API era completamente pública: cualquier persona que conociera la URL podía consultar, crear, modificar o eliminar incidentes. El objetivo de esta tarea fue reforzar la plataforma con controles de seguridad en varias capas e integrar un cliente web en HTML y JavaScript que consuma la API de forma segura.

Las mejoras aplicadas fueron: autenticación con JSON Web Tokens (JWT), autorización por roles (ADMIN, SUPERVISOR y ANALISTA), contraseñas protegidas con bcrypt, CORS restringido, cabeceras seguras con Helmet, limitación de peticiones, validación con Zod y limpieza de entradas con sanitize-html. Todo se verificó con catorce pruebas automatizadas.

2. Fallos encontrados (riesgos iniciales)

Tomando como referencia el OWASP Top 10, se identificaron estos riesgos:

- **Falta de autenticación y de control de acceso (A01 y A07).** Ningún endpoint pedía identificar al usuario. Una petición `DELETE /api/incidentes/1` enviada desde cualquier cliente borraba el registro. Tampoco había roles para separar funciones.
- **Exposición de datos (A02).** Los incidentes podían leerse libremente y no estaba definido cómo guardar credenciales. Almacenar contraseñas en texto plano habría expuesto a todos los usuarios ante una fuga.
- **Validación incompleta (A03).** Zod verificaba tipos y longitudes, pero aceptaba HTML y JavaScript. Un título `<script>alert(1)</script>` se guardaba tal cual y podía provocar un XSS almacenado.
- **CORS permisivo y configuración insegura (A05).** No había política CORS explícita ni cabeceras de seguridad, y la cabecera `X-Powered-By: Express` revelaba la tecnología del servidor.
- **Sin límites de uso.** El login quedaba expuesto a fuerza bruta y la API aceptaba cuerpos de cualquier tamaño.

3. Mejoras realizadas

3.1 JWT, roles y bcrypt

Se creó el endpoint `POST /api/auth/login`. Un esquema Zod estricto exige un correo válido y una contraseña de 8 a 72 caracteres (72 es el máximo que procesa bcrypt) y rechaza campos inesperados, como un `rol` enviado desde el cliente.

Los usuarios están desacoplados del código: las contraseñas se leen de variables de entorno y se persisten mediante siembra (`prisma/seed.js`) con hashes **bcrypt (costo 10)**, sin credenciales en texto plano. Si un correo no existe, se compara contra un hash señuelo para evitar ataques de tiempo (timing attacks), respondiendo siempre "Credenciales inválidas".

Con credenciales correctas se firma un JWT con algoritmo **HS256 fijado explícitamente**, emisor definido, expiración de una hora y solo identificador, correo y rol. El secreto `JWT_SECRET` se valida al arrancar: si falta o tiene menos de 32 caracteres, la aplicación no inicia.

La autorización usa dos middlewares. `autenticar` verifica la cabecera `Authorization: Bearer <token>` (firma, algoritmo, emisor y expiración) y responde **401** si algo falla. `permitir(...roles)` responde **403** si el rol no está autorizado. Las rutas quedaron así: `GET` para cualquier usuario autenticado, `POST` para ANALISTA y ADMIN, `PUT`

para SUPERVISOR y ADMIN, y `DELETE` solo para ADMIN. Una prueba confirmó que cambiar el rol dentro del token invalida la firma y el servidor lo rechaza.

3.2 CORS, Helmet y HTTPS

CORS solo admite los orígenes de `CLIENT_ORIGIN` (`http://localhost:5500`), los métodos GET, POST, PUT y DELETE y las cabeceras `Content-Type` y `Authorization` . Un origen no autorizado recibe 403 sin `Access-Control-Allow-Origin` , por lo que el navegador bloquea la respuesta. El cliente se sirve en un puerto distinto al de la API para probar esta política en condiciones reales.

Helmet añade CSP, HSTS, `X-Content-Type-Options` y `X-Frame-Options` , y se desactivó `X-Powered-By` . Además se fijaron 300 peticiones cada 15 minutos por IP, 10 intentos fallidos de login en el mismo periodo y un máximo de 10 KB por cuerpo JSON.

HTTPS no se activó en el entorno local, pero es obligatorio en producción. El JWT viaja en cada petición y, sobre HTTP, cualquiera que intercepte la red (por ejemplo, en una Wi-Fi pública) podría robarlo y suplantar al usuario. TLS cifra el canal, protege la integridad de los mensajes y autentica al servidor. Lo recomendable es un proxy inverso (Nginx o un balanceador) con certificado válido que redirija todo el tráfico HTTP a HTTPS. La aplicación ya habilita `trust proxy` en producción, y la cabecera HSTS obliga al navegador a usar HTTPS.

3.3 Validación y sanitización como defensa en profundidad

Cada petición pasa por este orden: CORS → límite de peticiones → JSON → **sanitize-html** → autenticación → autorización → **Zod** → controlador. El sanitizador elimina todas las etiquetas HTML del cuerpo, descarta por completo el contenido de `<script>` y bloquea claves como `__proto__` . Las contraseñas no se modifican, porque solo se comparan con bcrypt. Después, Zod valida el texto ya limpio, así que un título formado solo por código malicioso queda vacío y se rechaza por longitud.

El cliente muestra todo el contenido con `textContent` y plantillas, nunca con `innerHTML` , y tiene su propia CSP. Por eso, aunque una capa fallara, el texto no se ejecutaría como código. El token se guarda en `sessionStorage` , que se borra al cerrar la pestaña, y el cliente vuelve al login ante un 401.

4. Conclusión

La arquitectura separa con claridad las responsabilidades: el cliente se ocupa de la presentación y el servidor de la seguridad. El cliente oculta los botones que el rol no puede usar, pero eso es solo una ayuda visual: la autorización real la decide el servidor en cada petición. Así lo demostraron los casos en que un ANALISTA intentó eliminar y un SUPERVISOR intentó crear, y ambos recibieron 403.

Los contratos entre cliente y servidor son explícitos: los esquemas Zod definen qué datos se aceptan, los códigos HTTP (400, 401, 403, 404, 429) indican cada tipo de error y el manejador centralizado no filtra trazas ni detalles de la base de datos. Con bcrypt, JWT firmado, roles, CORS restringido, cabeceras seguras, validación y sanitización, la plataforma pasó de no tener controles a contar con varias capas independientes que protegen la confidencialidad y la integridad de los incidentes. Las pruebas automatizadas permiten repetir esta verificación después de cualquier cambio.

Anexo Técnico: Registro de Pruebas y Evidencias

> **Metodología de Verificación:** Banco de 14 pruebas automatizadas sobre la API en ejecución mediante `npm test` (`tests/seguridad.test.js`), complementado con capturas de pantalla de la interacción visual e inspección de red.

A.1 Matriz de Usuarios y Credenciales de Prueba

Rol	Correo Electrónico	Contraseña	Permisos en API (/api/incidentes)
ADMIN	`admin@incidentes.com`	`Admin123!`	GET, POST, PUT, DELETE
SUPERVISOR	`supervisor@incidentes.com`	`Supervisor123!`	GET, PUT
ANALISTA	`analista@incidentes.com`	`Analista123!`	GET, POST

A.2 Resumen de Resultados de Pruebas (14/14 Casos Aprobados)

#	Caso de Prueba	Entrada / Método	Código Esperado	Resultado Obtenido	Estado
1	Acceso sin token	`GET /api/incidentes` (sin auth)	401 Unauthorized	401 - `UNAUTHORIZED`	**PASS**
2	Permisos insuficientes	`DELETE /api/incidentes/1` (Token ANALISTA)	403 Forbidden	403 - `FORBIDDEN`	**PASS**
3	Datos inválidos en login	`{"email":"correo-invalido","password":"123"}`	400 Bad Request	400 - Zod errors en email y password	**PASS**
4	Datos inválidos (Creación)	`{"titulo":123,"prioridad":"URGENTE"}`	400 Bad Request	400 - Validación de esquema Zod	**PASS**
5	Limpieza de entradas (XSS)	Título con `<script>alert(1)</script>`	201 Created	201 - Persistido sin etiquetas HTML	**PASS**
6	Login exitoso	Credenciales válidas de ANALISTA	200 OK + JWT	200 - Token Bearer emitido	**PASS**
7	Crear y Consultar	`POST` como ANALISTA, luego `GET`	201 Created + 200 OK	201 & Fila reflejada en tabla	**PASS**
8	Actualizar estado	`PUT /api/incidentes/:id` como SUPERVISOR	200 OK	200 - Estado: EN_PROCESO	**PASS**
9	Token manipulado (Firma)	JWT con payload alterado a ADMIN	401 Unauthorized	401 - `INVALID_TOKEN`	**PASS**
10	Contraseña incorrecta	Correo correcto y clave errónea	401 Unauthorized	401 - "Credenciales inválidas"	**PASS**
11	SUPERVISOR intenta crear	`POST` con token SUPERVISOR	403 Forbidden	403 - Acción denegada por RBAC	**PASS**
12	CORS restringido	`GET` con `Origin: http://sitio-malicioso.com`	403 Forbidden	403 - Sin cabecera ACAO	**PASS**
13	Cabeceras Helmet	Inspección de cabeceras HTTP	Headers seguros	CSP, HSTS, nosniff presentes	**PASS**
14	Eliminar incidente (ADMIN)	`DELETE /api/incidentes/:id` como ADMIN	200 OK	200 OK, borrado y 404 posterior	**PASS**

A.3 Evidencia Gráfica de la Interfaz Web y Red

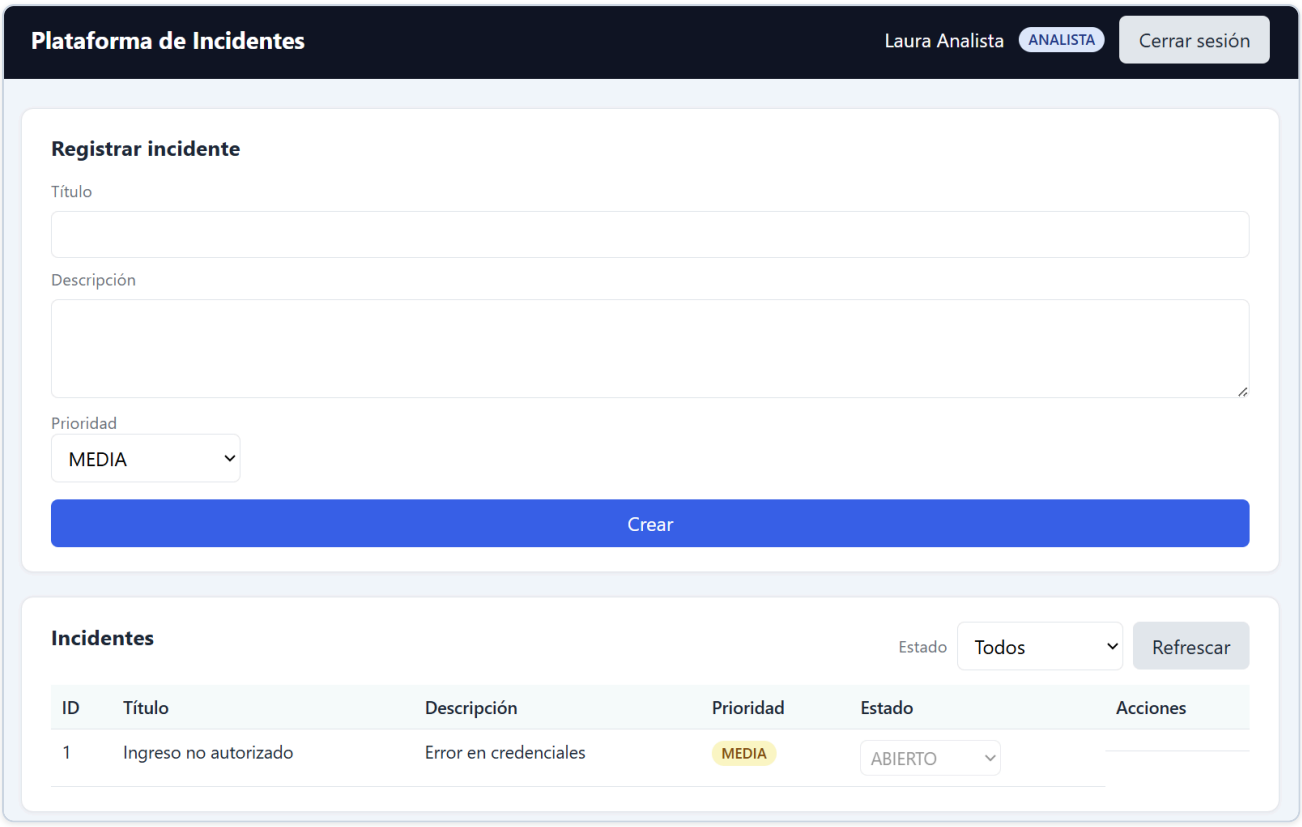


Figura 1: Formulario de inicio de sesión seguro con usuarios de demostración y validación en cliente.

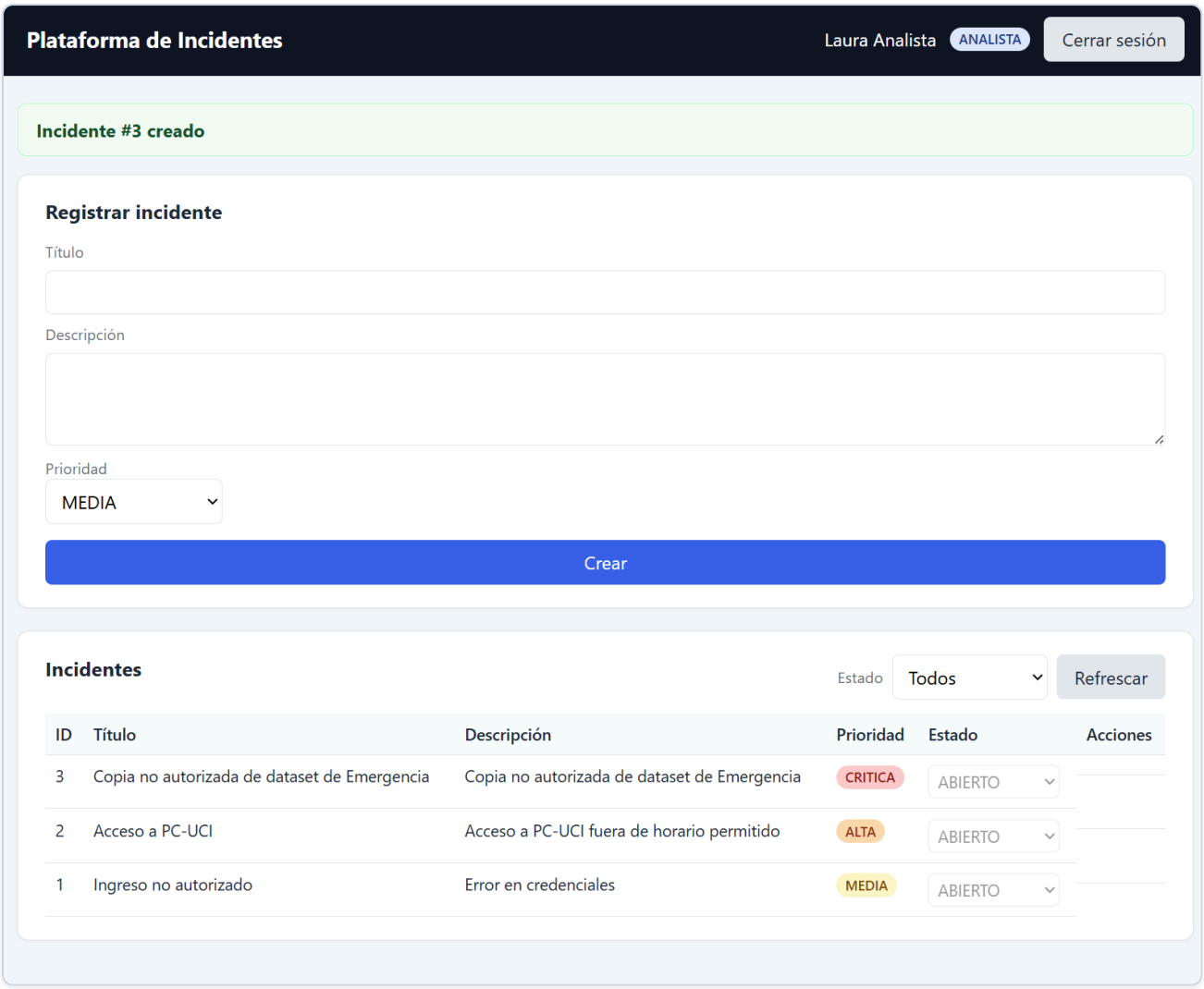


Figura 2: Respuesta HTTP 400 con desglose por campo ante correo mal formado y clave corta.



Figura 3: Consola de red (DevTools) comprobando el preflight OPTIONS de CORS y la cabecera Authorization: Bearer.

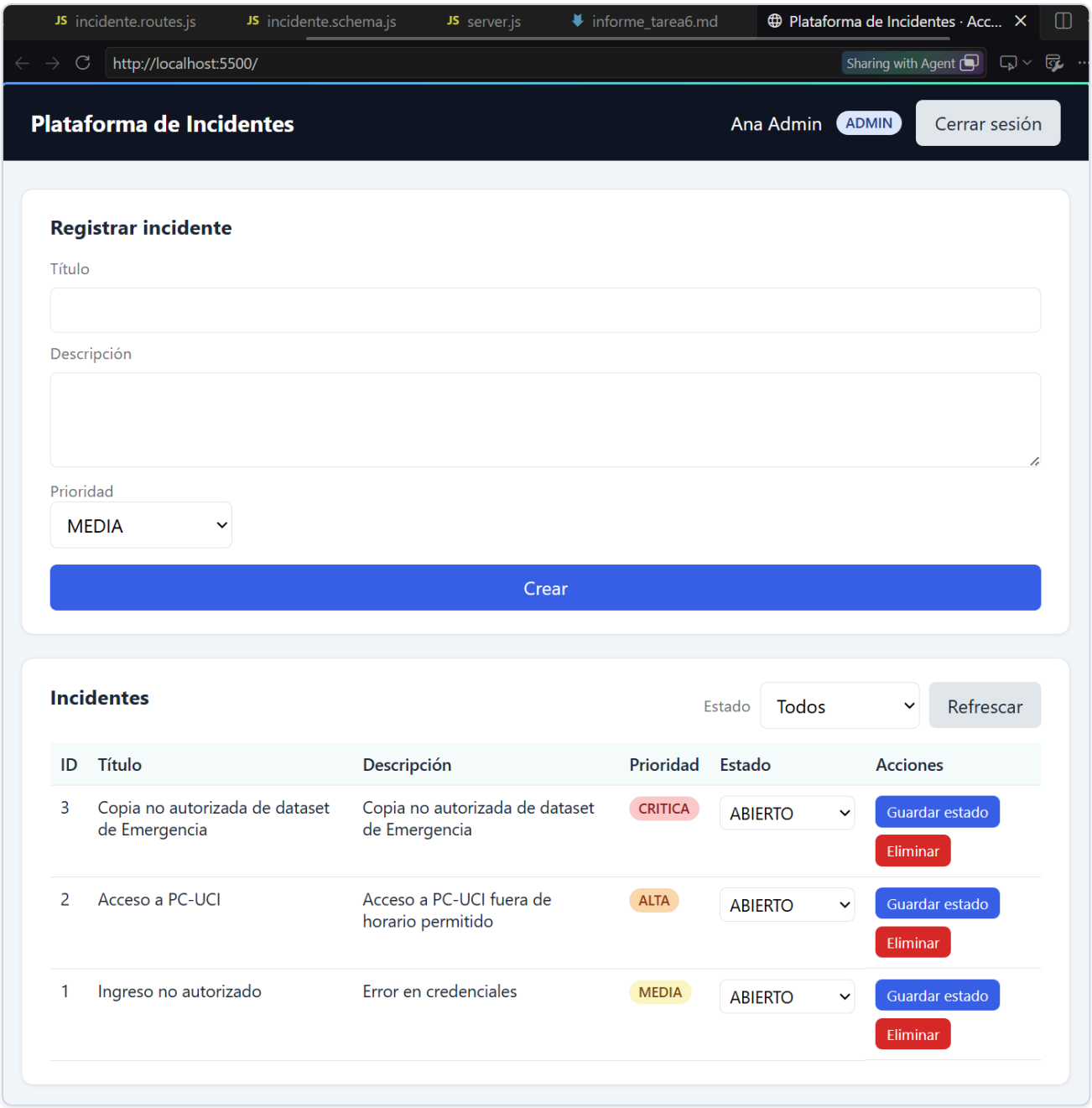


Figura 4: Interfaz autenticada como ANALISTA: permite registrar incidentes y oculta controles de eliminación.

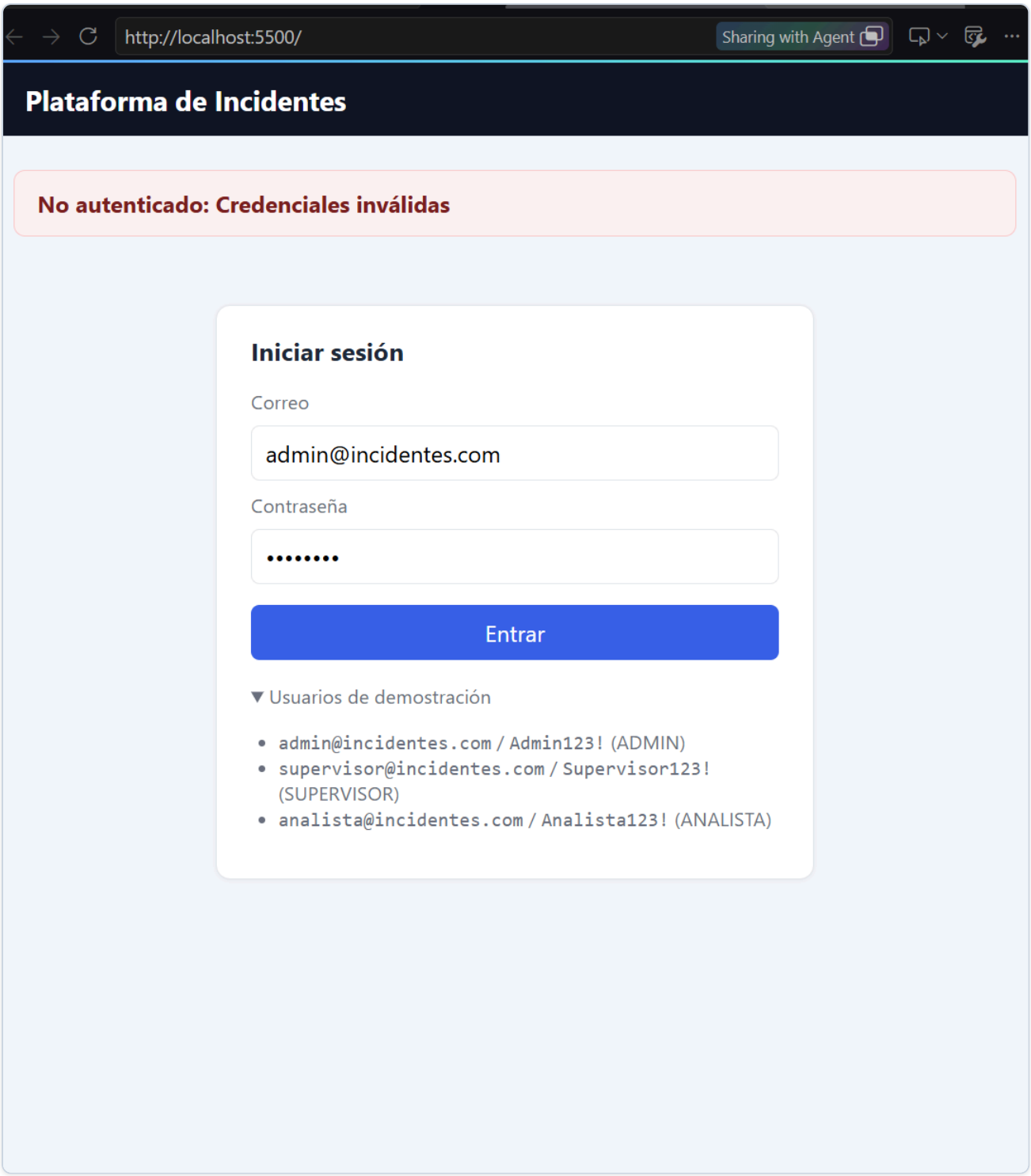


Figura 5: Interfaz autenticada como SUPERVISOR: control de estado habilitado y ejecución de actualización exitosa.

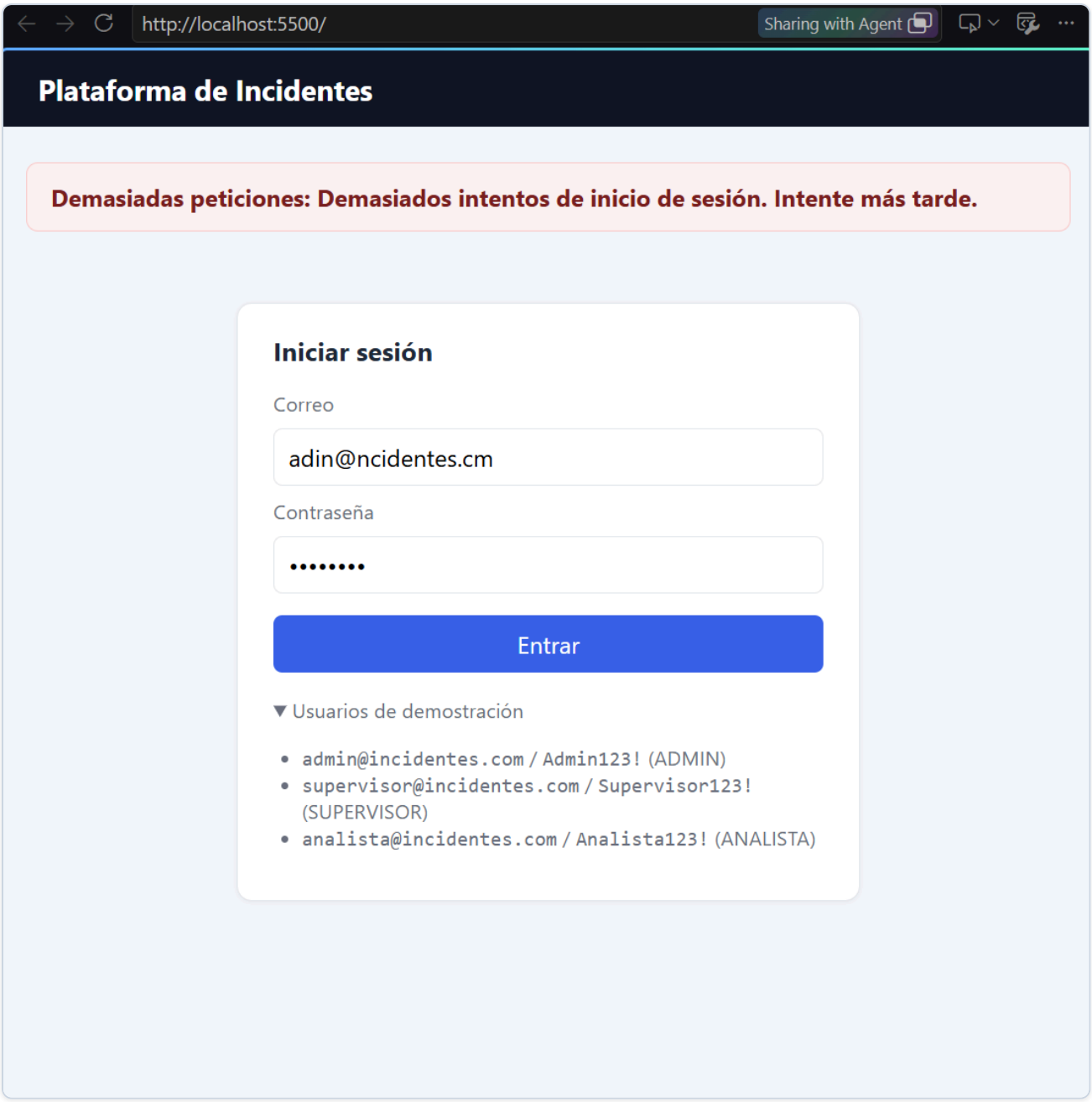


Figura 6: Interfaz con privilegios de ADMIN: control total CRUD y botón de eliminación habilitado.

A.4 Detalle Técnico de los Casos Obligatorios

Caso 1: Acceso sin token

Entrada: `GET /api/incidentes` (Sin cabecera Authorization)

```
HTTP 401 Unauthorized
{
  "success": false,
  "message": "Token de acceso requerido",
  "error": "UNAUTHORIZED"
}
```

Caso 2: Permisos insuficientes (ANALISTA intenta DELETE)

Entrada: `DELETE /api/incidentes/1` con cabecera `Authorization: Bearer <Token_ANALISTA>`

```
HTTP 403 Forbidden
{
```

```
  "success": false,
  "message": "Acceso denegado: se requiere uno de los roles [ADMIN]",
  "error": "FORBIDDEN"
}
```

Caso 3: Datos inválidos en login

Entrada: `POST /api/auth/login` con cuerpo erróneo:

```
{ "email": "correo-sin-arroba", "password": "123" }

HTTP 400 Bad Request
{
  "success": false,
  "message": "Datos de entrada inválidos",
  "errors": [
    { "campo": "email", "mensaje": "El correo no tiene un formato válido" },
    { "campo": "password", "mensaje": "La contraseña debe tener al menos 8 caracteres" }
  ]
}
```

Caso 4: Datos inválidos en creación de incidente

Entrada: `POST /api/incidentes` con datos ausentes y tipos erróneos:

```
{ "titulo": 123, "prioridad": "URGENTE" }

HTTP 400 Bad Request
{
  "success": false,
  "message": "Datos de entrada inválidos",
  "errors": [
    { "campo": "titulo", "mensaje": "El título debe ser texto" },
    { "campo": "descripcion", "mensaje": "La descripción es obligatoria" },
    { "campo": "prioridad", "mensaje": "La prioridad debe ser BAJA, MEDIA, ALTA o CRITICA" }
  ]
}
```

Caso 5: Limpieza y Saneamiento de Entradas (XSS Prevention)

Entrada: `POST /api/incidentes` inyectando código HTML y script:

```
{
  "titulo": "<script>alert(\\"XSS\\")</script>Caída del servidor web",
  "descripcion": "<img src=x onerror=\\"alert(1)\\">El portal no responde <b>desde las 8:00</b>",
  "prioridad": "ALTA"
}

HTTP 201 Created (Persistido sin etiquetas)
{
  "id": 2,
  "titulo": "Caída del servidor web",
  "descripcion": "El portal no responde desde las 8:00",
  "prioridad": "ALTA",
  "estado": "ABIERTO"
}
```

Caso 6: Flujo exitoso de Login

Entrada: `POST /api/auth/login` con credenciales de `analista@incidentes.com`

```
HTTP 200 OK
{
  "success": true,
  "message": "Inicio de sesión exitoso",
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "tipo": "Bearer",
    "usuario": { "id": 3, "nombre": "Laura Analista", "email": "analista@incidentes.com", "rol": "ANALISTA" }
```

```
  }
}
```

Caso 7: Flujo exitoso de Crear y Consultar Incidente

Entrada: Creación como ANALISTA y listado con `GET /api/incidentes`

```
HTTP 201 Created (Creación de incidente)
{ "id": 3, "titulo": "Fallo en VPN corporativa", "prioridad": "CRITICA", "estado": "ABIERTO" }

HTTP 200 OK (Consulta GET)
Total: 3 incidentes. La fila se renderizó en la tabla HTML mediante textContent de forma segura.
```

Caso 8: Flujo exitoso de Actualización por SUPERVISOR

Entrada: `PUT /api/incidentes/3` con `{ "estado": "EN_PROCESO" }` portando token con rol SUPERVISOR.

```
HTTP 200 OK
{
  "success": true,
  "message": "Incidente actualizado exitosamente",
  "data": { "id": 3, "titulo": "Fallo en VPN corporativa", "estado": "EN_PROCESO" }
}
```